ONTARIO TECH UNIVERSITY
FACULTY OF SCIENCE, COMPUTER SCIENCE

Project Group: 27 April 05, 2026

Simulation and Modeling Course Project: Traffic Intersection Simulation

by

Harsh Panchal

Abstract: This project looks into how to optimize a 4-way traffic intersection, by building a 2D simulation that mixes continuous physics with random discrete events. The main focus is to see how changing traffic light timings and the amount of incoming traffic affect the performance of the intersection.

Repository:
https://github.com/HarshPanchal01/Traffic-Intersection-Simulation

Execution:
Run `python3 main.py` to view the visual simulation.
Run `python3 main.py --headless` to run batch experiments and generate data plots.

1 Introduction

The goal of this project was to simulate a 4-way traffic intersection and evaluate the parameters required to maintain optimal traffic flow. Traffic intersections act as complex systems where continuous physical movements (vehicle kinematics) intersect with discrete events, such as traffic signal state changes and random vehicle arrivals.

By developing this simulation, we can study how these systems reach an equilibrium, and observe their behavior when pushed beyond their normal carrying capacity. Among the considered outcomes were:

1. The impact of varying green light phase durations on average wait times, queue lengths, and vehicle throughput. The reasoning behind this was to try and identify optimal signal timings that minimize overall delays.

2. The system's response to increasing volumes of incoming traffic. This aimed to determine the intersection's maximum throughput capacity and identify the exact point at which queue lengths grow uncontrollably, resulting in systemic failure.

To address these inquiries, the simulation actively tracks each vehicle's speed, wait time, and position. It then aggregates this data to evaluate the overall efficiency of the intersection under various configurations.

2 Related Work

The conceptual foundation of this project heavily relies on integrating several concepts covered in the course into a single, cohesive model:

- **Continuous Systems:** The foundational kinematics of the vehicles, specifically their acceleration, velocity, and position, which draw inspiration from continuous system models, such as projectile motion and mass-spring systems. To achieve this, Ordinary Differential Equations (ODEs) were used to mathematically update vehicle movement over continuous time.

- **Discrete Event Systems (DES):** The traffic light's state machine and the resulting queueing behavior of vehicles waiting at red lights directly reflect

DES methodologies. In this context, the intersection effectively acts as a server that processes waiting entities (the vehicles) from their respective queues.

- **Random Processes:** To ensure the environment behaved organically rather than being purely deterministic, the generation of incoming traffic is modeled as a Poisson process utilizing inverse transform sampling. This statistical technique mimics unpredictable natural occurrences, similar to radioactive decay, allowing for highly realistic and randomized vehicle arrival rates.

- **Collision Detection:** Finally, the underlying logic required to maintain physical realism and prevent vehicles from crashing into one another within the intersection utilizes standard distance-based collision detection and spatial awareness algorithms.

3 Methodology

The simulation is built using Python. Pygame was used for the visual side of things and SciPy and NumPy to handle the math. It works as a hybrid model that mixes continuous physics with random discrete events.

3.1 Continuous Kinematics

Vehicle movement is modeled using a system of first-order ordinary differential equations (ODEs). A vehicle's state is defined as $\mathrm{s} = [x, v]$, where x is its 1D position along its path and v is its speed. The math for this comes from standard Newtonian physics [2]:

$$\frac{dx}{dt} = v, \quad \frac{dv}{dt} = a \tag{1}$$

Here, a is the acceleration chosen by the driver's decision-making logic. To keep the physics accurate and avoid math errors when cars slow down, SciPy's Runge-Kutta 8th order (`dop853`) solver was used to integrate these equations.

When a car gets close to a red light or a slower vehicle, the driver calculates a safe braking distance. This uses a standard physics equation based on the work-energy theorem [2]:

$$s = \frac{v_{initial}^2}{2 \times a_{braking}} \tag{2}$$

If the distance to the obstacle is within this braking limit plus a small safety buffer, the car hits the brakes as hard as it can.

3.2 Stochastic Spawning

To make the traffic flow look natural, cars do not just spawn at fixed times. Instead, Poisson distribution was applied. The Inverse Transform Sampling method was utilized to generate random time gaps between arrivals.

If you want an average arrival rate of λ (vehicles per second) and have a random number U between 0 and 1, the time t until the next car spawns is calculated using this formula [3]:

$$t = -\frac{1}{\lambda} \ln(1 - U) \tag{3}$$

This method does a really good job of recreating the bunching effect you see in real-world traffic.

3.3 Collision Avoidance and Intersection Logic

There were issues with cars overlapping while turning, so a fix was implemented by unrolling the curved paths into straight 1D lines in the code. The distance to the car in front is then just the difference in their 1D x positions along that arc.

Additionally a cross-traffic blocking rule was added to prevent gridlock. Before a car enters the intersection on a green light, it checks the main intersection box. If cars going the other way are stuck and moving very slowly (less than 15 pixels per second), the car waits at the stop line, just like real traffic laws dictate.

4 Results

The data was gathered by running the simulation in headless mode. This gave enough results to be statistically significant without slowing down with all of the visual rendering. Two main experiments were conducted, testing the system in 5-minute simulated blocks.

4.1 Experiment 1: Varying Green Light Duration

For the first experiment, the traffic arrival rate locked at 0.5 vehicles per second, and the green light duration was changed from 5 to 60 seconds.

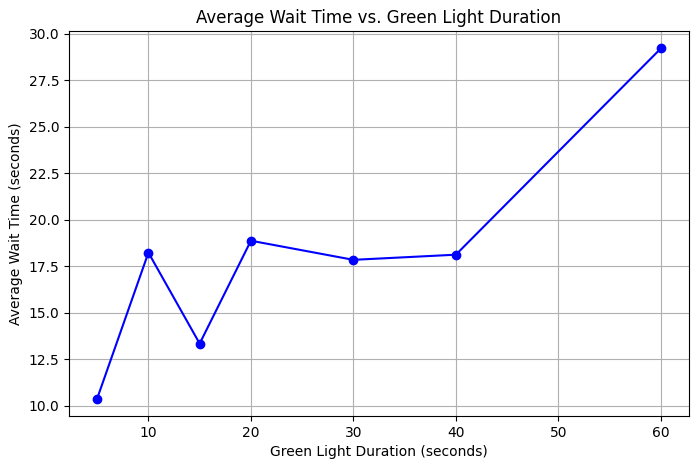

Figure 1: Average Wait Time vs. Green Light Duration

The data shows that really short green lights (around 5 seconds) actually lead to the lowest average wait times, sitting right around 10 seconds for this amount of traffic. As the green lights last longer, the wait times mostly went up, hitting almost 30 seconds when the light was set to 60 seconds. This happens because long green lights force the cross-traffic to just sit there waiting, which brings down the overall average.

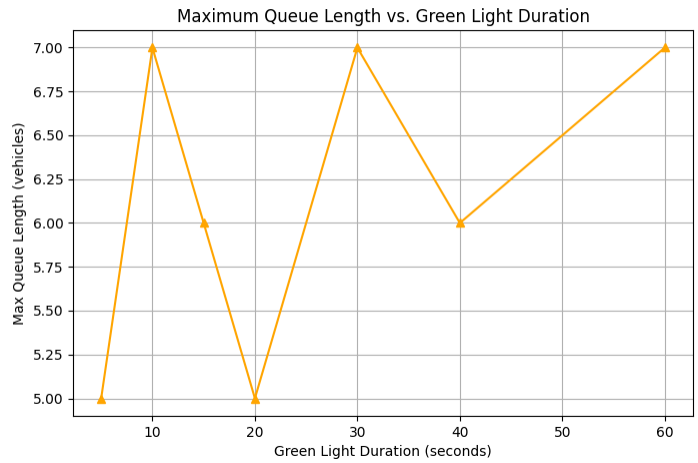

Figure 2: Maximum Queue Length vs. Green Light Duration

The queue length data jumps around a bit. The maximum lines were usually between 5 and 7 cars for the different light setups. This says that even though shorter cycles can lower average wait times, they can sometimes cause really fast backups if a bunch of cars all show up right when the light turns red.

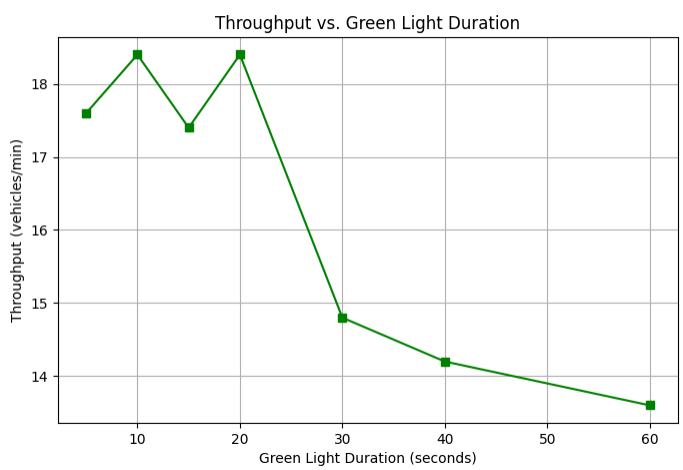

Figure 3: Throughput vs. Green Light Duration

Looking at the throughput, the system performed best when the lights were between 10 and 20 seconds long, pushing through about 18.5 cars per minute. Anything longer than 20 seconds caused a huge drop in throughput, falling to around 13.6 cars per minute at the 60-second mark. This basically confirms that really long light cycles are just inefficient and lower the amount of traffic the intersection can handle.

4.2 Experiment 2: Varying Arrival Rate

In the second experiment, the green light is locked at 20 seconds and traffic is slowly increased from 0.1 to 1.25 vehicles per second.

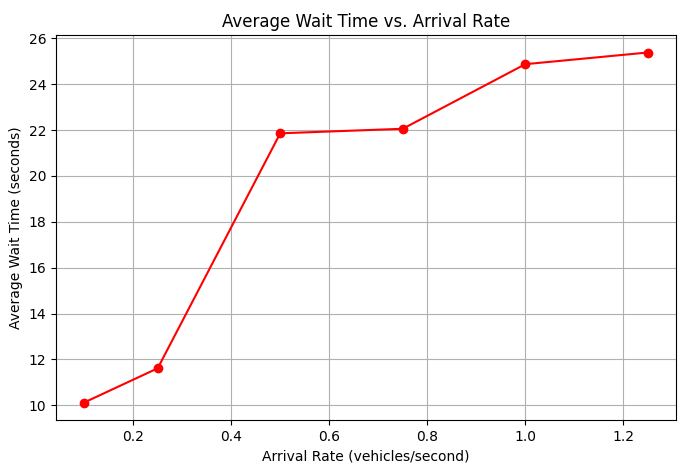

Figure 4: Average Wait Time vs. Arrival Rate

The average wait times show a really clear turning point. When traffic is light (0.1 to 0.25 cars per second), wait times stay pretty flat at around 10 to 11 seconds. But once the rate gets close to 0.5 cars per second, the wait time doubles to about 22 seconds. This shows exactly when congestion starts, as the system just cannot clear the lines of cars in a single light cycle anymore.

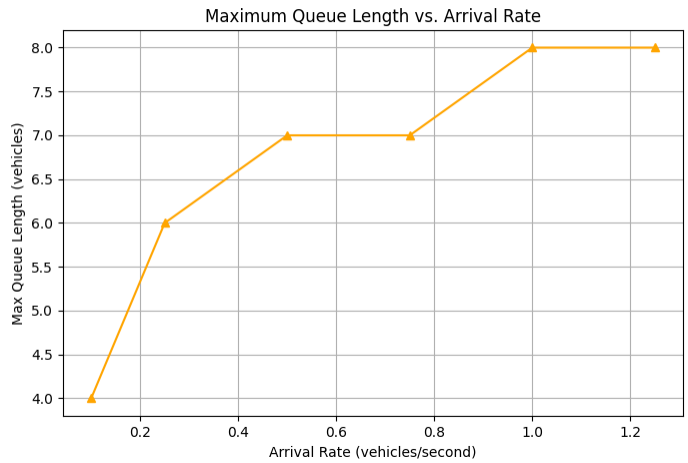

Figure 5: Maximum Queue Length vs. Arrival Rate

The maximum line lengths grow right alongside the traffic rate, going from 4 cars at 0.1 cars per second up to a flat-line of 8 cars when the rate hits 1.0 cars per second and higher. This flat-line shows that the physical space or the light cycle is totally maxed out at that point.

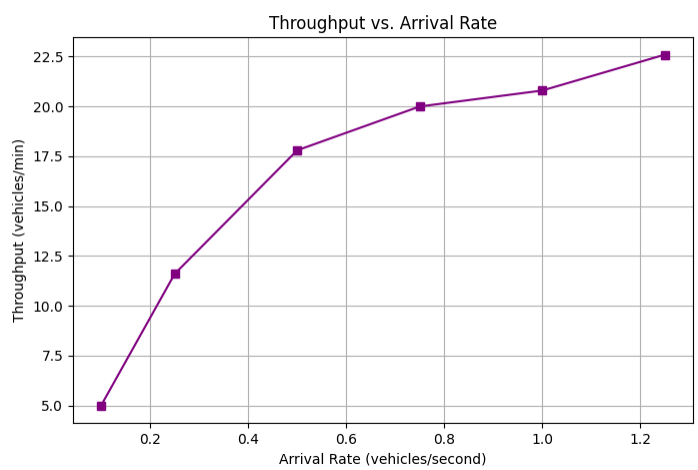

Figure 6: Throughput vs. Arrival Rate

The throughput data paints a really clear picture of how much the system can hold. At first, the throughput climbs right along with the amount of incoming traffic. But after hitting 0.75 cars per second, the growth slows way down, hitting a hard ceiling at about 22.5 cars per minute. At this breaking point, the physical limits of how fast cars can accelerate and the safety gaps they need to keep just stop the intersection from handling any more traffic. In the real world, this is where the traffic jam would just keep growing forever.

5 Conclusion

Overall, this simulation models how a 4-way intersection actually works. By combining continuous physics for the cars and the random Poisson math for how they spawn, the model gives a pretty realistic look at traffic waves and congestion limits.

The test results show that traffic light timing has to be balanced perfectly with how many cars are on the road. Keeping the green lights somewhere between 10 and 20 seconds seems to be the sweet spot for pushing the most cars through while keeping wait times low during normal traffic. And, the system definitely has a hard limit on what it can handle, which sits right around 22 cars per minute. If traffic gets heavier than that, the wait times just blow up and the lines get completely out of control. These findings prove how ideas about discrete event systems and continuous modeling can be used to study real-world infrastructure.

References

[1] Qureshi, F. Z. (2026). *Simulation and Modeling (CSCI 3010U) Lecture Notes*. Ontario Tech University. `https://csundergrad.science.uoit.ca/courses/csci3010u/latest/`

[2] Halliday, D., Resnick, R., & Walker, J. (2013). *Fundamentals of Physics* (10th ed.). John Wiley & Sons. `https://ia801601.us.archive.org/24/items/fundamentals-of-physics-textbook/fundamentals-of-physics-textbook.pdf`

[3] Ross, S. M. (2012). *Simulation* (5th ed.). Academic Press. `https://www.academia.edu/34737123/Simulation_Fifth_Edition_Sheldon_M_Ross_2012`